Principal - Self-Assessment

This self-assessment is designed to help us validate the accuracy and complexity of the new competency profiles. Your responses will not be used for performance evaluation or HR decisions. Instead, your feedback will help us:

- Confirm whether each competency is appropriately defined
- Ensure the expected complexity matches real-world experience
- Identify areas where the profile may need refinement or clarification

Your honest input is essential and greatly appreciated, as it will directly contribute to improving the competency framework for everyone.

Awareness Level Definitions (0–3):

- **Level 0:** *I Don't Know*
  - You have no awareness of this technology or concept and cannot describe it.
- **Level 1:** *I Heard About It*
  - You recognize the term and understand it at a high level, but have no hands-on experience.
- **Level 2:** *I Know the Foundation*
  - You understand the fundamentals and can perform basic or guided tasks.
- **Level 3:** *I Know It Very Well*
  - You can apply the concept independently, troubleshoot issues, and work confidently.

Hi, Ahmed. When you submit this form, the owner will see your name and email address.

* Required

1. Core Competency: Retrieval & Knowledge Integration - Sub-competency: Vector Embeddings & Search – Objective: Can implement vector embedding generation and similarity search systems – Breakdown: Generate embeddings from text or documents using embedding models; Store and index vectors in vector databases; Perform similarity search based on semantic meaning; Understand embedding dimensions and distance metrics; Optimize vector search performance *

   - ◯ I don't know
   - ◯ I heard about it
   - ◯ I know the foundation

- ◯ I know it very well

2. Core Competency: Retrieval & Knowledge Integration - Sub-competency: RAG Implementation – Objective: Can implement pro duction-grade RAG systems with measurable retrieval quality improvements – Breakdown: Learning how to use stored data or documents to improve model answers using tools that create and search embeddings.; Understand the concept of RAG and how it grounds model responses in real data; Upload or connect files and content sources for retrieval; Use tools or APIs to generate and store embeddings from text or documents; Search and retrieve relevant information based on meaning (not keywords); Inject retrieved data from vector DB into model prompts for more accurate answers; Keep the database updated when new files are added *

- ◯ I don't know
- ◯ I heard about it
- ◯ I know the foundation
- ◯ I know it very well

3. Core Competency: Agents & Orchestration - Sub-competency: Agent Concepts & Theory – Objective: Can understand when and why to use autonomous agents versus direct model calls – Breakdown: Understanding the foundations of AI agents.; Core concepts: agent, goal, action, observation, and context; Difference between stateless prompts and stateful agent loops; Agent autonomy and decision-making; When agents add value vs. when simple prompts suffice; Agent limitations and failure modes *

- ◯ I don't know
- ◯ I heard about it
- ◯ I know the foundation
- ◯ I know it very well

4. Core Competency: Design Patterns - Sub-competency: Scalability Patterns – Objective: Can implement patterns to support system scalability – Breakdown: Implement queue-based load leveling; Design caching strategies (cache-aside) *

- ◯ I don't know
- ◯ I heard about it
- ◯ I know the foundation
- ◯ I know it very well

5. Core Competency: Design Patterns - Sub-competency: Resilience Patterns – Objective: Can implement patterns to improve system resilience and reliability – Breakdown: Implements retry pattern with exponential backoff; Configure timeout policies *

- ◯ I don't know
- ◯ I heard about it
- ◯ I know the foundation
- ◯ I know it very well

6. Core Competency: Architectural Frameworks - Sub-competency: Clean Architecture – Objective: Can design systems using Clean Architecture principles – Breakdown: Structure applications with dependency inversion; Define independent layers (entities, use cases, interfaces); Design for testability and maintainability; Isolate business logic from frameworks; Apply SOLID principles at architecture level; Design for technology independence *

- ◯ I don't know
- ◯ I heard about it
- ◯ I know the foundation

- ◯ I know it very well

7. Core Competency: Distributed Systems - Sub-competency: Distributed Systems Basics – Objective: Can implement basic distributed system components – Breakdown: Differentiate between distributed system communication approaches (sync vs async) *

- ◯ I don't know
- ◯ I heard about it
- ◯ I know the foundation
- ◯ I know it very well

8. Core Competency: Distributed Systems - Sub-competency: Trade-offs and Challenges – Objective: Can evaluate distributed system trade-offs and design decisions – Breakdown: Analyze latency vs throughput trade-offs; Evaluate consistency vs availability decisions; Balance complexity vs benefits *

- ◯ I don't know
- ◯ I heard about it
- ◯ I know the foundation
- ◯ I know it very well

9. Core Competency: Code Design & OOP - Sub-competency: Inversion of Control (IoC) & Dependency Injection – Objective: Can architect systems using IoC principles and DI containers – Breakdown: Design for testability through DI; Understand service locator vs DI *

- ◯ I don't know
- ◯ I heard about it
- ◯ I know the foundation

- ◯ I know it very well

10. Core Competency: API & Service Design - Sub-competency: API Design & Versioning – Objective: Can design scalable APIs with proper versioning strategies – Breakdown: Design consistent API contracts; Implement API versioning (URL, header, query); Handle breaking vs non-breaking changes; Design for backward compatibility; Document APIs comprehensively; Apply API design best practices *

- ◯ I don't know
- ◯ I heard about it
- ◯ I know the foundation
- ◯ I know it very well

11. Core Competency: Exception Governance - Sub-competency: Advanced Exception Handling – Objective: Can implement sophisticated exception handling strategies – Breakdown: Handle async exceptions correctly; Implement exception policies; Create global exception interceptors; Design custom exception types; Log and track exceptions systematically *

- ◯ I don't know
- ◯ I heard about it
- ◯ I know the foundation
- ◯ I know it very well

12. Core Competency: Exception Governance - Sub-competency: Global Exception Handling – Objective: Can implement centralized exception handling across application – Breakdown: Create global exception middleware; Handle exceptions consistently; Return appropriate error responses; Log exceptions centrally; Avoid exception leakage to clients; Map exceptions to HTTP status codes *

- ◯ I don't know
- ◯ I heard about it
- ◯ I know the foundation
- ◯ I know it very well

13. Core Competency: Configuration Management - Sub-competency: Environment Configuration – Objective: Can manage application configuration across environments – Breakdown: Use environment variables effectively; Implement feature flags; Use external configuration stores; Manage secrets securely; Support multiple environments (dev, test, prod); Hot-reload configuration changes *

- ◯ I don't know
- ◯ I heard about it
- ◯ I know the foundation
- ◯ I know it very well

14. Core Competency: Resilience & Performance - Sub-competency: Background Jobs – Objective: Can implement and manage background job processing – Breakdown: Create hosted services or background workers; Schedule recurring jobs; Handle long-running tasks; Manage job queues; Implement retry logic for failed jobs; Monitor job execution *

- ◯ I don't know
- ◯ I heard about it
- ◯ I know the foundation
- ◯ I know it very well

15. Core Competency: Resilience & Performance - Sub-competency: Thread Pools & Worker Threads – Objective: Can optimize application performance using thread pools – Breakdown: Understand thread pool architecture; Configure thread pool settings; Use worker threads effectively; Avoid thread pool starvation; Monitor thread pool metrics; Choose between thread pool and dedicated threads *

- ◯ I don't know
- ◯ I heard about it
- ◯ I know the foundation
- ◯ I know it very well

16. Core Competency: Data Access & ORM - Sub-competency: Advanced Querying – Objective: Can write complex database queries including aggregations – Breakdown: Write aggregated queries (SUM, AVG, COUNT, GROUP BY); Use subqueries effectively; Implement complex joins; Optimize query performance; Use query hints when necessary; Understand query execution plans *

- ◯ I don't know
- ◯ I heard about it
- ◯ I know the foundation
- ◯ I know it very well

17. Core Competency: Data Access & ORM - Sub-competency: Transaction Management – Objective: Can implement proper transaction handling for data consistency – Breakdown: Use transactions for atomic operations; Understand ACID properties; Implement transaction scopes; Handle distributed transactions; Manage transaction isolation levels; Rollback on failures *

- ◯ I don't know
- ◯ I heard about it

- ◯ I know the foundation
- ◯ I know it very well

18. Core Competency: Core Technologies & Infrastructure - Sub-competency: Cloud Storage – Objective: Can implement cloud storage solutions with appropriate blob types and data transfer strategies – Breakdown: Select appropriate storage types (block, append, page blobs); Implement blob storage operations; Configure data archiving policies; Use SDK/tools for data transfer *

- ◯ I don't know
- ◯ I heard about it
- ◯ I know the foundation
- ◯ I know it very well

19. Core Competency: Core Technologies & Infrastructure - Sub-competency: Cloud Queue Storage – Objective: Can implement message queuing systems for asynchronous processing – Breakdown: Implement enqueue/dequeue operations; Configure message lifecycle and visibility timeout; Implement polling mechanisms; Handle message expiration; Monitor queue performance and health; Apply queue limitations (message size, retention, throughput) *

- ◯ I don't know
- ◯ I heard about it
- ◯ I know the foundation
- ◯ I know it very well

20. Core Competency: Security & Identity Management - Sub-competency: Secrets & Key Management – Objective: Can implement secure secrets and key management for applications – Breakdown: Store and retrieve secrets programmatically; Integrate secrets management into CI/CD *

- ◯ I don't know
- ◯ I heard about it
- ◯ I know the foundation
- ◯ I know it very well

21. Core Competency: Monitoring & Management - Sub-competency: Monitoring Tools (Basics) – Objective: Can configure services to send logs and metrics to cloud monitoring platforms – Breakdown: Configure log collection from services; Send metrics to monitoring platforms; Create simple dashboards; Query logs and metrics *

- ◯ I don't know
- ◯ I heard about it
- ◯ I know the foundation
- ◯ I know it very well

22. Core Competency: Transactional Integrity & Storage - Sub-competency: Relational Databases & ACID Principles – Objective: Can implement systems with strong transactional guarantees – Breakdown: Apply ACID properties (Atomicity, Consistency, Isolation, Durability); Design transaction boundaries; Choose appropriate isolation levels; Handle concurrency and locking; Prevent deadlocks; Implement optimistic vs pessimistic locking; Design for data consistency *

- ◯ I don't know
- ◯ I heard about it
- ◯ I know the foundation
- ◯ I know it very well

23. Core Competency: Database Programming & Objects - Sub-competency: Materialized Views vs Views – Objective: Can evaluate and implement materialized views for performance optimization – Breakdown: Compare views vs materialized views; Design materialized views for complex queries; Configure refresh strategies (immediate, scheduled); Optimize query performance with materialized views *

- ◯ I don't know
- ◯ I heard about it
- ◯ I know the foundation
- ◯ I know it very well

24. Core Competency: Indexing, Optimization & Performance - Sub-competency: Execution Plans & Query Analysis – Objective: Can analyze and optimize query performance using execution plans – Breakdown: Read and interpret execution plans; Identify table scans and their impact; Analyze index scans vs index seeks; Identify missing indexes; Optimize join operations; Identify expensive operations (sorts, spools); Use query hints when appropriate; Monitor query performance metrics *

- ◯ I don't know
- ◯ I heard about it
- ◯ I know the foundation
- ◯ I know it very well

25. Core Competency: Containerization - Sub-competency: Docker Images & Containers – Objective: Can build Docker images and run containers – Breakdown: Write Dockerfiles; Build Docker images; Run containers from images; Manage container lifecycle (start, stop, remove); Inspect running containers; View container logs; Execute commands in containers; Tag and version images; Push images to registries *

- ◯ I don't know

- ◯ I heard about it
- ◯ I know the foundation
- ◯ I know it very well

26. Core Competency: CI/CD - Sub-competency: Build & Release Pipelines – Objective: Can create and manage automated build and release pipelines – Breakdown: Design pipeline workflows; Configure build triggers; Define build steps and tasks; Implement automated testing in pipeline; Configure artifact publishing; Create release pipelines; Implement deployment stages (dev, staging, prod) *

- ◯ I don't know
- ◯ I heard about it
- ◯ I know the foundation
- ◯ I know it very well

27. Core Competency: Frontend Frameworks - Sub-competency: Dependency Injection – Objective: Can use dependency injection in frontend frameworks – Breakdown: Understand DI concepts and benefits; Configure injectors and providers; Inject services into components; Implement different injection types; Manage service lifecycle; Apply loose coupling principles; Follow framework DI patterns *

- ◉ I don't know
- ◯ I heard about it
- ◯ I know the foundation
- ◯ I know it very well

28. Core Competency: Frontend Frameworks - Sub-competency: Component-Based Architecture – Objective: Can implement component-based architectures –

Breakdown: Design reusable components ; Implement component communication (props/events) ; Apply component composition patterns ; Use slots/content projection ; Implement controlled vs uncontrolled components *

- ◯ I don't know
- ◯ I heard about it
- ◯ I know the foundation
- ◯ I know it very well

29. Core Competency: JavaScript/TypeScript - Sub-competency: Reactive Programming (RxJS) – Objective: Can implement reactive programming patterns using RxJS – Breakdown: Apply operators for data transformation (map, filter, merge); Manage subscriptions and memory leaks; Handle streams of events; Understand event loop and async behavior; Implement reactive patterns; Debug observable chains *

- ◯ I don't know
- ◯ I heard about it
- ◯ I know the foundation
- ◯ I know it very well

30. Core Competency: JavaScript/TypeScript - Sub-competency: Decorators – Objective: Can use decorators for meta-programming in TypeScript – Breakdown: Understand decorator concepts and syntax; Apply class decorators; Apply method decorators; Apply property decorators; Apply parameter decorators; Create custom decorators; Use framework-specific decorators; Combine multiple decorators *

- ◯ I don't know
- ◯ I heard about it
- ◯ I know the foundation

- ◯ I know it very well

31. Core Competency: Authorization - Sub-competency: OAuth Interactive Flows – Objective: Can implement OAuth interactive authentication flows – Breakdown: Implement Authorization Code Flow; Implement PKCE for mobile/SPA apps; Implement Device Code Flow; Handle OAuth callbacks; Store and refresh access tokens; Implement OAuth error handling *

- ◯ I don't know
- ◯ I heard about it
- ◯ I know the foundation
- ◯ I know it very well

32. Core Competency: Cloud Security - Sub-competency: Secrets & Key Management Objective: Can implement secure secrets and key management – Breakdown: Create and store secrets in key vaults; Retrieve secrets programmatically; Set secret expiration dates *

- ◯ I don't know
- ◯ I heard about it
- ◯ I know the foundation
- ◯ I know it very well –

33. Core Competency: AI Security - Sub-competency: RAG Data Access Controls – Objective: Can implement access controls for RAG document retrieval – Breakdown: Design document-level access policies; Implement user-based document filtering; Configure role-based document access; Implement metadata-based access control; Filter retrieval results by permissions; Audit document access attempts; Handle unauthorized document access; Test RAG access control enforcement *

- ◯ I don't know

- ◯ I heard about it
- ◯ I know the foundation
- ◯ I know it very well

Microsoft 365

This content is created by the owner of the form. The data you submit will be sent to the form owner. Microsoft is not responsible for the privacy or security practices of its customers, including those of this form owner. Never give out your password.

Microsoft Forms | AI-Powered surveys, quizzes and polls Create my own form

Privacy and cookies | Terms of use